**UNIVERSIDADE DA REGIÃO DE JOINVILLE – UNIVILLE**
**ENGENHARIA DE SOFTWARE**

GUSTAVO FRATONI BOEING
CLEVERSON FILIPE CORDEIRO
RICHARD KAIAN BITENCOURT FERREIRA
ALAN DE OLIVEIRA GONÇALVES
KAUAN AMELIO DA SILVA DE MOURA
JUAN LUCAS SANTOS
KAUAN CHRISTOPHER THEILACKER

# O SUCESSO DO JSON NA ARQUITETURA REST

JOINVILLE-SC
2026

GUSTAVO FRATONI BOEING
CLEVERSON FILIPE CORDEIRO
RICHARD KAIAN BITENCOURT FERREIRA
ALAN DE OLIVEIRA GONÇALVES
KAUAN AMELIO DA SILVA DE MOURA
JUAN LUCAS SANTOS
KAUAN CHRISTOPHER THEILACKER

# O SUCESSO DO JSON NA ARQUITETURA REST

Trabalho apresentado à Universidade da Região de Joinville - UNIVILLE como requisito para entrega do trabalho a respeito de APIs RESTful.

Orientadora: Prof(a). Márcia

JOINVILLE-SC
2026

## RESUMO

Este trabalho discute o papel do formato JSON (*JavaScript Object Notation*) na arquitetura REST (*Representational State Transfer*) e os motivos que levaram à sua ampla adoção em relação ao XML na troca de dados entre aplicações web. A pesquisa bibliográfica apresenta os fundamentos dos serviços web, do protocolo HTTP e do estilo REST, além das características do XML e do JSON. Em seguida, é apresentado um estudo prático: um script em Python que consulta a API pública de CNPJ da CNPJá, converte a resposta JSON para XML, CSV e HTML e compara, entre dois formatos escolhidos pelo usuário, a quantidade de tokens de cada um com o tokenizador *tiktoken*. O código-fonte, seu funcionamento e os resultados da execução são apresentados e discutidos. A discussão indica que a simplicidade da sintaxe, a leitura direta por diferentes linguagens de programação e a menor verbosidade ajudam a explicar a ampla adoção do JSON em APIs RESTful, ainda que o XML continue relevante em contextos que exigem validação rígida por esquema.



**Palavras-chave**: API. JSON. REST. XML. Tokens.

**SUMÁRIO**


**1 INTRODUÇÃO** **5**

1.1 Formato XML 5

**2 DESENVOLVIMENTO** **7**

2.1 Arquitetura REST e protocolo HTTP 7

2.2 O formato JSON 7

2.3 JSON, XML e outros formatos 8

2.4 Estudo prático: comparação de tokens entre formatos 9

2.4.1 Tokens e motivação 9

2.4.2 Fonte dos dados 10

2.4.3 Ferramentas utilizadas 10

2.4.4 Funcionamento do script 10

2.4.5 Execução e resultados 13

2.4.6 Análise dos resultados 14

2.4.7 Limitações do experimento 15

**3 CONCLUSÃO** **16**

**REFERÊNCIAS** **17**

**APÊNDICE A – Código-fonte completo do script rest.py** **19**

# 1 INTRODUÇÃO

Os serviços web permitem a troca de informações e dados entre sistemas sem interação humana. A comunicação ocorre sobre protocolos da internet, como o HTTP (Hypertext Transfer Protocol) e o HTTPS (Hypertext Transfer Protocol Secure), por comandos como GET, POST, PUT, PATCH e DELETE [1, 2], e os dados em formatos XML e o JSON. Usar o próprio HTTP já resolve o problema de juntar vários mecanismos diferentes de troca de informação, e desta forma que se tornou a solução padrão para integrar sistemas. Neste cenário, destaca-se o REST (Representational State Transfer), proposto por Fielding e adotado pelas chamadas APIs RESTful. Uma parte expressiva dessas APIs adota o JSON (JavaScript Object Notation), cuja sintaxe simples e flexível favorece a comunicação entre serviços web. É o caso das APIs do WhatsApp Business e do Stripe. Esse predomínio, porém, é recente: antes dele, o formato principal era o XML.

Diante disto, este trabalho busca explicar por que o JSON se consolidou na arquitetura REST e mostrar, na prática, como o formato escolhido afeta o tamanho dos dados trocados (medido em tokens). Para isso, foi feita uma pesquisa bibliográfica em normas técnicas (RFCs e recomendações do W3C) e na documentação oficial de APIs, além de um estudo prático em Python que consulta a API pública da CNPJá, converte a resposta para JSON, XML, CSV e HTML e compara a contagem de tokens de dois desses formatos.

O trabalho se divide em três seções: a seção 2 traz a fundamentação teórica e o estudo prático, com código e resultados, e a seção 3 apresenta as considerações finais. O código-fonte completo está no Apêndice A.

## 1.1 Formato XML

O Formato XML (eXtensible Markup Language) foi o formato dominante na troca de dados entre sistemas antes do JSON. Derivado do SGML, tornou-se recomendação do W3C no fim dos anos 1990. Com a sua ascensão, surgiram os serviços web baseados em SOAP, protocolo de mensagens construído sobre XML, e naquele período o XML era a escolha natural para integrar sistemas diferentes.

O XML descreve os dados por meio de elementos delimitados por tags de abertura e fechamento, que aceitam atributos e espaços de nomes. Essa estrutura

permite representar documentos complexos e validá-los com rigor por meio de esquemas. Em contrapartida, a flexibilidade custa em verbosidade, tema retomado na seção 2.3.

# 2 DESENVOLVIMENTO

## 2.1 Arquitetura REST e protocolo HTTP

O REST foi apresentado por Fielding, em sua tese de doutorado, como um estilo arquitetural para sistemas distribuídos na web, definido por restrições: separação entre cliente e servidor, comunicação sem estado (stateless), cache, interface uniforme, organização em camadas e, opcionalmente, código sob demanda. Na prática, os serviços que seguem essas restrições são chamados de RESTful. Em uma API REST, cada recurso (um cliente, um pedido, uma empresa) é identificado por uma URL, e o cliente interage com ele pelos métodos do HTTP.
O Quadro 1 resume os principais:

Quadro 1 – Principais métodos HTTP utilizados em APIs REST

| **Método** | **Finalidade típica** |
|---|---|
| GET | Recupera a representação de um recurso, sem alterá-lo. |
| POST | Envia dados ao servidor, geralmente para criar um novo recurso. |
| PUT | Substitui por completo a representação de um recurso existente. |
| PATCH | Aplica uma alteração parcial a um recurso. |
| DELETE | Remove um recurso. |

Fonte: Adaptado de Fielding, Nottingham e Reschke e de Dusseault e Snell [1, 2].

O REST não impõe um formato de dados específico. O servidor envia ao cliente uma representação do recurso, e o formato é informado nos cabeçalhos HTTP, como Content-Type e Accept. Assim, a mesma API poderia responder em JSON, XML, CSV ou HTML. Escolher o formato é, portanto, uma decisão de projeto, e é nela que o JSON se destacou. O HTTP também informa o resultado de cada requisição por códigos de status: a família 2xx indica sucesso, a 4xx indica erro do cliente (como o 400, requisição inválida) e a 5xx indica falha no servidor. Essa classificação é usada no tratamento de erros do script da seção 2.4.

## 2.2 O formato JSON

O JSON é um formato de texto para representar dados estruturados. Sua sintaxe deriva da forma como objetos são escritos em JavaScript, mas o formato independe de linguagem e foi popularizado por Douglas Crockford a partir dos anos

2000. Hoje é padronizado pela RFC 8259 e pela norma ECMA-404, e seu tipo de mídia é application/json. Possui apenas duas estruturas: o objeto, formado por pares de nome e valor, e a lista (array), uma sequência ordenada de valores. Os valores podem ser texto (string), número, verdadeiro ou falso (true/false), nulo (null), outro objeto ou outra lista. Quando trocado entre sistemas que não pertencem a um mesmo ambiente fechado, o texto JSON deve usar a codificação UTF-8.

Essas características explicam boa parte do sucesso do formato em APIs REST. A sintaxe é pequena e fácil de ler; objetos e listas correspondem diretamente aos dicionários e listas da maioria das linguagens (em Python, dict e list), então ler uma resposta se resume a chamar uma função, como orjson.loads no script deste trabalho; por ser nativo do JavaScript, foi adotado rapidamente em aplicações executadas no navegador; e serviços de grande alcance, como as APIs do Stripe e do WhatsApp Business, usam JSON em suas respostas. O formato também tem limitações: não aceita comentários, não possui tipo próprio para datas (geralmente enviadas como texto) e não traz validação embutida, embora existam especificações complementares, como o JSON Schema.

## 2.3 JSON, XML e outros formatos

O XML foi, por muitos anos, o formato padrão dos serviços web, principalmente nos baseados em SOAP. Sua força está em descrever documentos complexos, com atributos, espaços de nomes e validação rigorosa por esquemas. Em contrapartida, é verboso: cada valor exige uma tag de abertura e outra de fechamento.

O Quadro 2 compara os dois formatos:

Quadro 2 – Comparação entre JSON e XML

| **Critério** | **JSON** | **XML** |
|---|---|---|
| Sintaxe | Objetos entre chaves, listas entre colchetes e pares nome/valor. | Elementos delimitados por *tags* de abertura e fechamento, com atributos opcionais. |
| Verbosidade | Menor: o nome do campo aparece uma vez para cada valor. | Maior: o nome do campo aparece na *tag* de abertura e na de fechamento. |
| Tipos de dados | Texto, número, booleano, nulo, objeto e lista. | Todo conteúdo é texto; os tipos são definidos por esquema (XSD). |

| Critério | JSON | XML |
|---|---|---|
| Validação | Feita por especificações complementares, como o JSON Schema. | DTD e XML Schema (XSD) fazem parte do ecossistema XML. |
| Leitura em código | Mapeia diretamente para dicionários e listas da linguagem. | Exige percorrer uma árvore de elementos (DOM, SAX etc.). |

Fonte: Elaborado pelos autores com base em Bray e Bray *et al.* [4, 7].

Os Quadros 3 e 4 mostram o mesmo dado nos dois formatos. Repare que o nome de cada campo aparece duas vezes no XML e apenas uma vez no JSON.

Quadro 3 – Exemplo de dado em JSON

```
{
  "pessoa": {
    "nome": "Ana",
    "cidade": "Joinville"
  }
}
```

Fonte: Elaborado pelos autores (2026).

Quadro 4 – Mesmo dado em XML

```
<pessoa>
  <nome>Ana</nome>
  <cidade>Joinville</cidade>
</pessoa>
```

Fonte: Elaborado pelos autores (2026).

Além de JSON e XML, o script também gera CSV e HTML. O CSV representa dados em linhas e colunas, sem hierarquia, então dados aninhados, como os retornados pela API, precisam ser "achatados" antes da conversão. O HTML, por sua vez, é uma linguagem de marcação voltada à apresentação de páginas, e não à troca de dados entre sistemas; entra na comparação para mostrar o peso de uma marcação de tabela sobre os mesmos dados.

## 2.4 Estudo prático: comparação de tokens entre formatos

### *2.4.1 Tokens e motivação*

Modelos de linguagem de grande escala não processam o texto caractere por caractere, mas em unidades chamadas tokens, pedaços de palavras gerados por algoritmos como o byte pair encoding. O tamanho do texto que o modelo aceita e o custo de uso costumam ser medidos em tokens. Por isso, o formato escolhido para entregar dados a um modelo importa: dois formatos com o mesmo conteúdo podem

consumir quantidades diferentes de tokens. A biblioteca de código aberto tiktoken implementa os tokenizadores dos modelos da OpenAI e permite contar tokens localmente.

Esse é o ponto de partida do script: para um mesmo conjunto de dados, quanto "pesa", em tokens, cada formato?

### *2.4.2 Fonte dos dados*

Os dados vêm da API pública da CNPJá, que consulta informações cadastrais de empresas pelo CNPJ (Cadastro Nacional da Pessoa Jurídica). A consulta usa o método GET no endereço `https://open.cnpja.com/office/:cnpj`, sem cadastro nem autenticação, e retorna um objeto JSON com os dados da empresa. A API pública é limitada a 5 consultas por minuto por endereço IP. Como a resposta contém objetos aninhados e listas, ela é adequada para testar as conversões entre formatos.

### *2.4.3 Ferramentas utilizadas*

O script foi escrito em Python e usa as seguintes bibliotecas:

a) `requests`: realiza a requisição HTTP GET à API;

b) `orjson`: lê a resposta JSON e serializa dados para texto;

c) `lxml`: constrói e formata a saída em XML;

d) `pandas`: achata o JSON em tabela com `json_normalize` e gera o CSV e o HTML;

e) `tiktoken`: conta os tokens de cada saída;

f) `re`: módulo da biblioteca padrão usado para limpar o que o usuário digita.

### *2.4.4 Funcionamento do script*

O programa segue o fluxo descrito a seguir:

a) exibe a mensagem de abertura (`opening`);

b) solicita o CNPJ e dois formatos distintos (`format_choice` e `ask_format`), limpando o CNPJ para que contenha apenas dígitos (`clean_input`);

c) consulta a API com GET e verifica o status da resposta (Quadro 9);

d) converte o JSON recebido para os dois formatos escolhidos (`convert_json`);

e) conta os tokens de cada resultado com o tokenizador do modelo `gpt-4o` (`count_token`), usando o `cl100k_base` como alternativa;

f) imprime os resultados na tela.

A função `count_token` (Quadro 5) obtém o codificador do modelo informado com `tiktoken.encoding_for_model`. Se o modelo não for reconhecido (`KeyError`), usa o codificador `cl100k_base`. O texto é transformado em uma lista de tokens, e o tamanho dessa lista é o resultado.

Quadro 5 – Função de contagem de tokens

```
def count_token(text: str, model: str = "gpt-4o") -> int:
    try:
        encoding = tiktoken.encoding_for_model(model)
    except KeyError:
        encoding = tiktoken.get_encoding("cl100k_base")

    token_list = encoding.encode(text)

    return len(token_list)
```

Fonte: Código-fonte do script rest.py (2026).

O CSV e o HTML precisam de dados em formato de tabela. A função `flatten_dict` (Quadro 6) usa `pandas.json_normalize` para transformar objetos aninhados em colunas com nomes compostos (como `objeto.campo`). Listas não cabem em uma célula, por isso, elas são convertidas em texto JSON com `orjson.dumps`.

Quadro 6 – Função que achata o JSON em tabela

```
def flatten_dict(json_text: dict):
    flattened_dict = pd.json_normalize([json_text] if isinstance(json_text, dict) else
json_text)
    for col in flattened_dict.columns:
        if flattened_dict[col].apply(lambda x: isinstance(x, list)).any():
            flattened_dict[col] = flattened_dict[col].apply(lambda x:
orjson.dumps(x).decode("utf-8") if isinstance(x, list) else x)
    return flattened_dict
```

Fonte: Código-fonte do script rest.py (2026).

A função recursiva `convert_json_to_xml` (Quadro 7) percorre o JSON e cria elementos XML com o `lxml`. Cada chave de um objeto vira um elemento, e valores simples viram o texto do elemento. Cada item de uma lista vira um elemento filho cujo nome é o da *tag* pai no singular (removendo o “s” final) ou, se a *tag* não

termina em “s”, o nome da *tag* pai seguido de `Son` e do índice do item. O elemento raiz é `pessoaJuridica`.

Quadro 7 – Função que converte JSON em XML

```
def convert_json_to_xml(father_element, json_text):
    if isinstance(json_text, dict):
        for key,value in json_text.items():
            son_element = etree.SubElement(father_element, str(key))
            if isinstance(value, (dict, list)):
                convert_json_to_xml(son_element, value)
            else:
                son_element.text = str(value)
    elif isinstance(json_text, list):
        father_tag_name = father_element.tag
        i = 0
        for value in json_text:
            singular_tag = father_tag_name[:-1] if father_tag_name.endswith('s') else
f"{father_tag_name}Son{i}"
            son_element = etree.SubElement(father_element, singular_tag)
            convert_json_to_xml(son_element, value)
            i += 1
    return father_element
```

Fonte: Código-fonte do script rest.py (2026).

A função `convert_json` (Quadro 8) funciona como um despachante: de acordo com o formato escolhido, devolve o JSON original, o XML, o CSV ou o HTML, usando a estrutura `match`/`case`, disponível a partir do Python 3.10.

Quadro 8 – Função que escolhe a conversão

```
def convert_json(json_text: dict, format: str):
    flattened_dict = None
    if format == "csv" or "html":
        flattened_dict = flatten_dict(json_text)


    match format:
        case "json":
            return json_text
        case "xml":
            return etree.tostring(convert_json_to_xml(etree.Element("pessoaJuridica"),
json_text), encoding="unicode", pretty_print=True)
        case "csv":
            return convert_json_to_csv(flattened_dict)
        case "html":
            return convert_json_to_html(flattened_dict)
        case _:
            print("\nFormato fora do padrão!\n")
            return None
```

Fonte: Código-fonte do script rest.py (2026).

O trecho principal (Quadro 9) monta a URL da API, faz a requisição e trata os erros: o status 400 indica CNPJ inválido e oferece a opção de reiniciar; os demais status abaixo de 500 encerram o programa por requisição fora do padrão; e status 500 ou superior indica indisponibilidade do servidor. Em caso de sucesso, o JSON é lido, convertido e os tokens são contados. O uso de aspas duplas dentro de f-strings nesse trecho exige Python 3.12 ou superior.

Quadro 9 – Trecho principal: requisição, tratamento de erros e contagem

```
opening()
while True:
    user_information = format_choice()
    base_url = "https://open.cnpja.com/office"
    url = f"{base_url}/{user_information["cnpj"]}"
    try:
        response = requests.get(url)
        response.raise_for_status()
    except requests.exceptions.HTTPError as error:
        status_code = error.response.status_code
        if status_code < 500:
            if status_code == 400:
                print("\nCNPJ digitado inválido! Tente novamente...\n")
                choice = input("\nDeseja reiniciar a aplicação? Digite (1) para sim e (0)
para não: ")
                if choice == '1':
                    continue
                elif choice == '0':
                    print("Encerrando...")
                    break
                else:
                    continue
            else:
                print("Requisição da API fora do padrão esperado, encerrando a
aplicação...")
                break
        else:
            print("Servidor inoperante ou sem resposta, encerrando a aplicação...")
            break
    else:
        response_json = orjson.loads(response.content)
        first_format = convert_json(response_json, user_information["first_format"])
        second_format = convert_json(response_json, user_information["second_format"])
        first_format_token = count_token(orjson.dumps(first_format).decode('utf-8'))
        second_format_token = count_token(orjson.dumps(second_format).decode('utf-8'))
        print("\n\n\n\n")
        print(f"Resultados da contagem de tokens do formato
{user_information["first_format"]}: {first_format_token}")
        print(f"Resultados da contagem de tokens do formato
{user_information["second_format"]}: {second_format_token}")
        break
```

Fonte: Código-fonte do script rest.py (2026).

### *2.4.5 Execução e resultados*

O script é executado com o comando `python rest.py`. Após a mensagem de abertura, o programa solicita o CNPJ (que pode ser digitado com pontos, barra e traço, pois a função `clean_input` mantém apenas os dígitos), o primeiro formato e o segundo formato, escolhidos por número de 1 a 4. Não é permitido repetir na segunda escolha o formato da primeira. Em seguida, o programa consulta a API, converte a resposta e imprime a contagem de tokens de cada formato, como mostra o Quadro 10.

Quadro 10 – Exemplo de interação com o script no terminal

```
Este é um script utilizado para comparar tokens

Digite o CNPJ desejado: 84.714.682/0001-94
Escolha entre os seguintes formatos:
 JSON(1)
 XML(2)
 CSV(3)
 HTML(4)

Digite o número correspondente ao primeiro formato desejado: 1
[lista de formatos exibida novamente]
Escolha anterior: json
Digite o número correspondente ao segundo formato desejado: 2

Resultados da contagem de tokens do formato json: 449
Resultados da contagem de tokens do formato xml: 940
```

Fonte: Saída do script rest.py (2026). Valores entre colchetes devem ser substituídos pelos obtidos na execução.

Como o script compara apenas dois formatos por execução, foram feitas três execuções com o mesmo CNPJ (JSON e XML, JSON e CSV, JSON e HTML) para montar a Tabela 1. A API pública permite até 5 consultas por minuto por endereço IP, limite suficiente para esse procedimento.

Tabela 1 – Contagem de tokens por formato (tokenizador do modelo gpt-4o)

| Formato | Tokens | Variação em relação ao JSON |
|---|---|---|
| JSON | 449 | — |
| XML | 940 | 491 |
| CSV | 562 | 113 |
| HTML | 1093 | 644 |

Fonte: Dados da pesquisa (2026). CNPJ consultado: 84.714.682/0001-94 (FURJ).

### *2.4.6 Análise dos resultados*

Do ponto de vista da estrutura, é esperado que o XML produza mais tokens que o JSON, pois cada valor é cercado por uma *tag* de abertura e outra de fechamento, repetindo o nome do campo duas vezes, enquanto o JSON repete o nome apenas uma vez. O HTML gerado pelo `pandas` segue lógica parecida, com marcações de tabela, linhas e células. O CSV, por escrever os nomes das colunas uma única vez no cabeçalho, tende a ser o mais compacto para dados achatados, ao custo de perder a hierarquia original. Todavia, a conversão para CSV não é

perfeita e por conta da repetição de aspas e a adaptação de características JSON incompatíveis com CSV, ele acaba perdendo sua vantagem teórica. Se a API suportasse CSV nativamente na requisição o cenário seria provavelmente muito diferente.

### *2.4.7 Limitações do experimento*

Os resultados devem ser lidos com alguns cuidados. Primeiro, o script serializa cada saída com `orjson.dumps` antes de contar os tokens; para os formatos de texto (XML, CSV e HTML), isso acrescenta aspas e caracteres de escape (como `\n` e `\t"`) ao texto, o que pode aumentar um pouco a contagem em relação ao texto original. Segundo, a tokenização depende do modelo: outros modelos podem gerar contagens diferentes. Terceiro, o resultado depende do CNPJ consultado, pois empresas com mais sócios e atividades geram respostas maiores. Quarto, o CSV e o HTML perdem a estrutura hierárquica, e as listas viram texto JSON dentro das células. Por fim, a API pública limita as consultas a 5 por minuto por IP.

## 3 CONCLUSÃO

Este trabalho mostrou que o sucesso do JSON na arquitetura REST não vem de uma imposição do estilo arquitetural, já que o REST não exige um formato específico, mas de fatores práticos: sintaxe pequena e legível, correspondência direta com as estruturas de dados das linguagens, suporte nativo no JavaScript e adoção por APIs de grande alcance.

O XML, que antecedeu o JSON como formato predominante, continua válido quando é preciso validação rigorosa por esquema, atributos e espaços de nomes ou integração com sistemas baseados em SOAP. Sua principal desvantagem é a verbosidade, já que os nomes dos campos se repetem nas tags de abertura e de fechamento.

No estudo prático, o script em Python consultou uma API REST real, converteu a resposta para quatro formatos e mediu o peso de cada um em tokens. Os resultados da Tabela 1 indicam que XML e HTML consumiram mais tokens que o JSON, enquanto o CSV ficou em um nível intermediário por conta das limitações da conversão.

# REFERÊNCIAS

[1] FIELDING, Roy T.; NOTTINGHAM, Mark; RESCHKE, Julian (ed.). **HTTP semantics**. [S. l.]: IETF, jun. 2022. (RFC 9110). Disponível em: https://www.rfc-editor.org/info/rfc9110. Acesso em: 4 out. 2026.

[2] DUSSEAULT, Lisa; SNELL, James M. **PATCH method for HTTP**. [S. l.]: IETF, mar. 2010. (RFC 5789). Disponível em: https://www.rfc-editor.org/info/rfc5789. Acesso em: 4 out. 2026.

[3] FIELDING, Roy Thomas. **Architectural styles and the design of network-based software architectures**. 2000. Tese (Doutorado em Informação e Ciência da Computação) – University of California, Irvine, 2000. Disponível em: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm. Acesso em: 4 out. 2026.

[4] BRAY, Tim (ed.). **The JavaScript Object Notation (JSON) data interchange format**. [S. l.]: IETF, dez. 2017. (RFC 8259). Disponível em: https://www.rfc-editor.org/info/rfc8259. Acesso em: 4 out. 2026.

[5] META. **WhatsApp Business Platform**: Cloud API. [S. l.], [s.d.]. Disponível em: https://developers.facebook.com/docs/whatsapp/cloud-api. Acesso em: 4 out. 2026.

[6] STRIPE. **Stripe API reference**. [S. l.], [s.d.]. Disponível em: https://docs.stripe.com/api. Acesso em: 4 out. 2026.

[7] BRAY, Tim *et al.* (ed.). **Extensible Markup Language (XML) 1.0**. 5. ed. [S. l.]: W3C, 26 nov. 2008. W3C Recommendation. Disponível em: https://www.w3.org/TR/xml/. Acesso em: 4 out. 2026.

[8] GUDGIN, Martin *et al.* (ed.). **SOAP Version 1.2 Part 1**: messaging framework. 2. ed. [S. l.]: W3C, 27 abr. 2007. W3C Recommendation. Disponível em: https://www.w3.org/TR/soap12-part1/. Acesso em: 4 out. 2026.

[9] RICHARDSON, Leonard; RUBY, Sam. **RESTful web services**. Sebastopol: O'Reilly Media, 2007.

[10] ECMA INTERNATIONAL. **ECMA-404**: the JSON data interchange syntax. 2. ed. Genebra: Ecma International, dez. 2017. Disponível em: https://ecma-international.org/publications-and-standards/standards/ecma-404/. Acesso em: 4 out. 2026.

[11] CROCKFORD, Douglas. **Introducing JSON**. [S. l.], [s.d.]. Disponível em: https://www.json.org. Acesso em: 4 out. 2026.

[12] SHAFRANOVICH, Yakov. **Common format and MIME type for comma-separated values (CSV) files**. [S. l.]: IETF, out. 2005. (RFC 4180). Disponível em: https://www.rfc-editor.org/info/rfc4180. Acesso em: 4 out. 2026.

[13] SENNRICH, Rico; HADDOW, Barry; BIRCH, Alexandra. Neural machine translation of rare words with subword units. *In*: ANNUAL MEETING OF THE

ASSOCIATION FOR COMPUTATIONAL LINGUISTICS, 54., 2016, Berlim. **Proceedings** [...]. Stroudsburg: ACL, 2016. p. 1715-1725.

[14] OPENAI. **tiktoken**. [S. l.], [s.d.]. Disponível em: https://github.com/openai/tiktoken. Acesso em: 4 out. 2026.

[15] CNPJÁ. **API Pública**. [S. l.], [s.d.]. Disponível em: https://cnpja.com/api/open. Acesso em: 4 out. 2026.

[16] REQUESTS. **Requests**: HTTP for Humans. [S. l.], [s.d.]. Disponível em: https://requests.readthedocs.io. Acesso em: 4 out. 2026.

[17] ORJSON. **orjson**: fast, correct Python JSON library. [S. l.], [s.d.]. Disponível em: https://github.com/ijl/orjson. Acesso em: 4 out. 2026.

[18] LXML. **lxml**: XML and HTML with Python. [S. l.], [s.d.]. Disponível em: https://lxml.de. Acesso em: 4 out. 2026.

[19] PANDAS. **pandas.json_normalize**. [S. l.], [s.d.]. Disponível em: https://pandas.pydata.org/docs/reference/api/pandas.json_normalize.html. Acesso em: 4 out. 2026.

# APÊNDICE A – Código-fonte completo do script rest.py

```
#!/usr/bin/env python3
import re
import requests
import orjson
from lxml import etree
import pandas as pd
import tiktoken

def clean_input(text: str, option: str = "") -> str:
    if option == "numero":
        return re.sub(r"\D", "", text)
    else:
        return re.sub(r"[^a-zA-ZÀ-ÿ ]", "", text)

def show_formats(formats) -> None:
    print("Escolha entre os seguintes formatos:")
    i = 1
    for format in formats:
        print(f" {format.upper()}({i})")
        i += 1

def ask_format(formats: list, position: str, previous_choice: int = None) -> int:
    while True:
        choice = None
        try:
            show_formats(formats)
            if previous_choice:
                print(f"Escolha anterior: {formats[previous_choice-1]}")
            choice = int(input(f"\nDigite o número correspondente ao {position} formato
desejado: "))
            if choice == previous_choice:
                raise ValueError("\nEscolha já feita anteriormente, tente escolher outro
número\n")
            if choice != 1 and choice != 2 and choice != 3 and choice != 4:
                raise ValueError("\nO número não corresponde a nenhuma das alternativas,
tente novamente!\n")
        except ValueError as error:
            if choice == previous_choice and isinstance(choice, int):
                print(error)
            elif (choice != '1' or choice != '2' or choice != '3' or choice != '4') and
isinstance(choice, int):
                print(error)
            else:
                print(f"\nDigite um número inteiro válido entre 1 e {len(formats)}. Tente
novamente!\n")
                continue
        else:
            break
    return choice

def format_choice() -> dict:
    formats = ["json","xml","csv","html"]

    cnpj = clean_input(input("\nDigite o CNPJ desejado: "), "numero")
    first_choice = ask_format(formats, "primeiro")
    second_choice = ask_format(formats, "segundo", first_choice)

    return {"first_format": formats[first_choice-1], "second_format":
formats[second_choice-1], "cnpj": cnpj}

def opening():
    print("\nEste é um script utilizado para comparar tokens\n\n\n")

def count_token(text: str, model: str = "gpt-4o") -> int:
    try:
        encoding = tiktoken.encoding_for_model(model)
    except KeyError:
```

```
        encoding = tiktoken.get_encoding("cl100k_base")

    token_list = encoding.encode(text)

    return len(token_list)

def flatten_dict(json_text: dict):
    flattened_dict = pd.json_normalize([json_text] if isinstance(json_text, dict) else
json_text)
    for col in flattened_dict.columns:
        if flattened_dict[col].apply(lambda x: isinstance(x, list)).any():
            flattened_dict[col] = flattened_dict[col].apply(lambda x:
orjson.dumps(x).decode("utf-8") if isinstance(x, list) else x)
    return flattened_dict


def convert_json_to_xml(father_element, json_text):
    if isinstance(json_text, dict):
        for key,value in json_text.items():
            son_element = etree.SubElement(father_element, str(key))
            if isinstance(value, (dict, list)):
                convert_json_to_xml(son_element, value)
            else:
                son_element.text = str(value)
    elif isinstance(json_text, list):
        father_tag_name = father_element.tag
        i = 0
        for value in json_text:
            singular_tag = father_tag_name[:-1] if father_tag_name.endswith('s') else
f"{father_tag_name}Son{i}"
            son_element = etree.SubElement(father_element, singular_tag)
            convert_json_to_xml(son_element, value)
            i += 1
    return father_element


def convert_json_to_csv(json_text):
    return json_text.to_csv(index=False)


def convert_json_to_html(json_text):
    return json_text.to_html(index=False, classes="table")


def convert_json(json_text: dict, format: str):
    flattened_dict = None
    if format == "csv" or "html":
        flattened_dict = flatten_dict(json_text)


    match format:
        case "json":
            return json_text
        case "xml":
            return etree.tostring(convert_json_to_xml(etree.Element("pessoaJuridica"),
json_text), encoding="unicode", pretty_print=True)
        case "csv":
            return convert_json_to_csv(flattened_dict)
        case "html":
            return convert_json_to_html(flattened_dict)
        case _:
            print("\nFormato fora do padrão!\n")
            return None


opening()
while True:
    user_information = format_choice()
    base_url = "https://open.cnpja.com/office"
    url = f"{base_url}/{user_information["cnpj"]}"
    try:
        response = requests.get(url)
        response.raise_for_status()
    except requests.exceptions.HTTPError as error:
        status_code = error.response.status_code
        if status_code < 500:
            if status_code == 400:
```

```
                print("\nCNPJ digitado inválido! Tente novamente...\n")
                choice = input("\nDeseja reiniciar a aplicação? Digite (1) para sim e (0)
para não: ")
                if choice == '1':
                    continue
                elif choice == '0':
                    print("Encerrando...")
                    break
                else:
                    continue
            else:
                print("Requisição da API fora do padrão esperado, encerrando a
aplicação...")
                break
        else:
            print("Servidor inoperante ou sem resposta, encerrando a aplicação...")
            break
    else:
        response_json = orjson.loads(response.content)
        first_format = convert_json(response_json, user_information["first_format"])
        second_format = convert_json(response_json, user_information["second_format"])
        first_format_token = count_token(orjson.dumps(first_format).decode('utf-8'))
        second_format_token = count_token(orjson.dumps(second_format).decode('utf-8'))
        print("\n\n\n\n")
        print(f"Resultados da contagem de tokens do formato
{user_information["first_format"]}: {first_format_token}")
        print(f"Resultados da contagem de tokens do formato
{user_information["second_format"]}: {second_format_token}")
        break
```

Fonte: Código-fonte do script rest.py (2026).